

Manual Completo: Instalación Paso a Paso de Oracle XE

OBJETIVO

Descargar la imagen de Oracle XE en Docker.

PRERREQUISITOS

- Docker Desktop instalado y ejecutándose
- Conexión a internet para descargar la imagen
- Mínimo 4GB de RAM disponibles

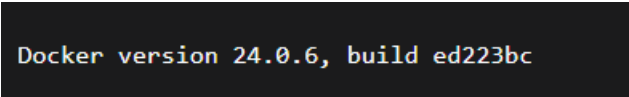
PASO 1: PREPARACIÓN DEL ENTORNO

Abrir Símbolo del Sistema (CMD) como Administrador

Verificar que Docker esté funcionando

```
docker --version
```

Salida esperada:



```
Docker version 24.0.6, build ed223bc
```

```
docker system info
```

Verificar que esté en estado: Running

PASO 2: LIMPIAR INSTALACIONES PREVIAS (OPCIONAL)

Verificar si existe el contenedor anterior

```
docker ps -a | findstr oracle-prueba
```

Si existe, detener y eliminar

```
docker stop oracle-prueba
```

```
docker rm oracle-prueba
```

Limpiar imágenes no utilizadas

`docker system prune -f`

PASO 3: DESCARGAR LA IMAGEN DE ORACLE XE

Ejecutar descarga de la imagen

`docker pull gvenzl/oracle-xe:latest`

- ✓ `docker pull` -> Le dice a Docker que descargue una imagen desde el repositorio de Docker Hub(u otro registro configurado)
- ✓ `gvenzl/oracle-xe` -> Es el nombre de la imagen. En este caso corresponde a la imagen de Oracle Database Express Edition(XE) publicada por Gerald Venzl, que es una versión ligera y gratuita de Oracle Database.
- ✓ `:latest` -> Es la etiqueta(tag) de la imagen. Latest indica que descargará la versión más reciente que haya disponible en el repositorio(si no se pone una etiqueta por defecto también se usa latest)

Progreso esperado de la descarga

```
latest: Pulling from gvenzl/oracle-xe
Digest: sha256:9d516e3912ca5bfd75ad4b6fea1351085b83b527c0db0ab7962706c154b04caa
Status: Downloaded newer image for gvenzl/oracle-xe:latest
```

PASO 4: VERIFICAR LA DESCARGA

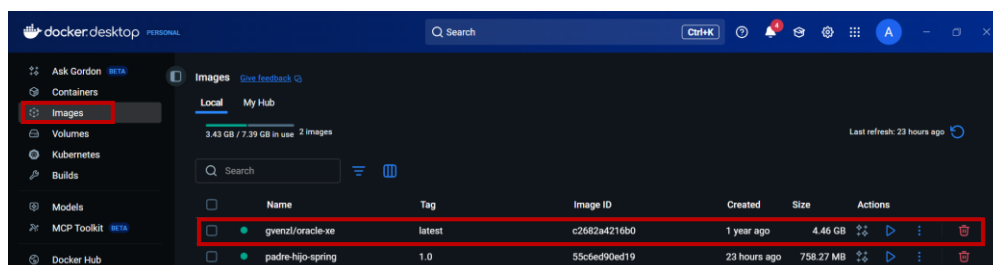
Listar imágenes descargadas

`docker images gvenzl/oracle-xe`

Salida esperada:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
gvenzl/oracle-xe	latest	a1b2c3d4e5f6	2 weeks ago	2.5GB

En Docker desktop debería verse así.



Verificar detalles de la imagen

```
docker image inspect gvenzl/oracle-xe:latest --format "Tamaño: {{.Size}} bytes | Creado: {{.Created}}"
```

PASO 5: CREAR Y EJECUTAR EL CONTENEDOR

```
docker run -d --name oracle-prueba -p 1522:1521 -e ORACLE_PASSWORD=12345 gvenzl/oracle-xe:latest
```

- ✓ `docker run` -> Crea y arranca un contenedor a partir de la imagen especificada.
- ✓ `-d` -> lo ejecuta en modo detached, es decir, en segundo plano.
- ✓ `--name Oracle-prueba` -> asigna el nombre Oracle-prueba al contenedor(para que no tengas que usar el ID largo del contenedor al administrarlo).
- ✓ `-p 1522:1521` -> mapea el puerto del contenedor al host:
 - 1521 es el puerto interno donde corre Oracle Database
 - 1522 es el puerto en tu maquina local por el que vas a conectarte
- ✓ `-e ORACLE_PASSWORD=12345` -> Define la variable de entorno ORACLE_PASSWORD que estable la contraseña del usuario administrador(SYS, SYSTEM, PDBADMIN)
- ✓ `gvenzl/oracle-xe:latest` -> es la imagen que se descargo

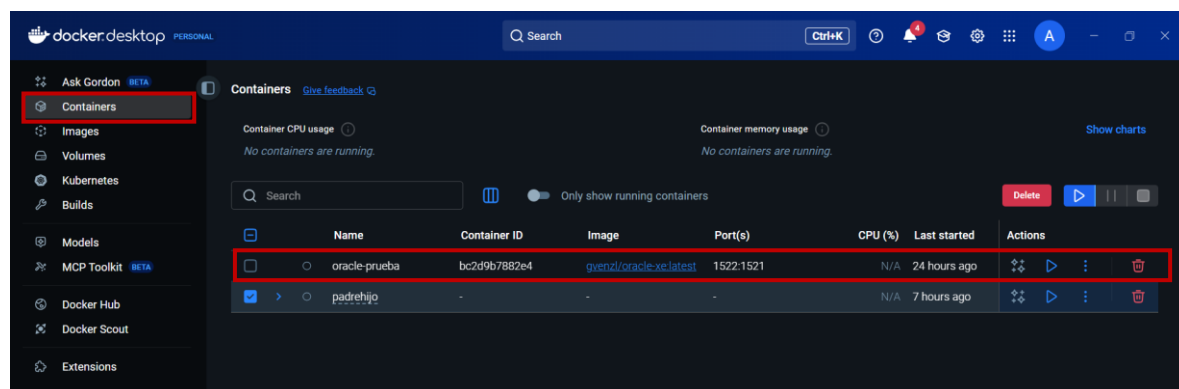
5.2 Verificar que el contenedor se creó

```
docker ps -a | findstr oracle-prueba
```

Salida esperada:

CONTAINER ID	IMAGE	STATUS	PORTS
a1b2c3d4e5f6	gvenzl/oracle-xe:latest	Created	

En Docker desktop



PASO 6: MONITOREAR LA INICIALIZACIÓN

Ver logs en tiempo real

```
docker logs -f oracle-prueba
```

Esperar señales de progreso:

```
✓ [1/5] - Container starting up...
✓ [2/5] - Uncompressing database files...
✓ [3/5] - Starting Oracle listener...
✓ [4/5] - Opening database...
✓ [5/5] - DATABASE IS READY TO USE!
```

Tiempos estimados:

- Descompresión de archivos: 2-3 minutos
- Inicialización de BD: 1-2 minutos
- Total: 3-5 minutos

PASO 7: VERIFICACIÓN FINAL

Comprobar estado del contenedor

```
docker ps
```

Salida esperada:

NAMES	STATUS	PORTS
oracle-prueba	Up	1522/tcp

Probar conexión a la base de datos

```
docker exec -it oracle-prueba sqlplus system/12345@XE
```

Comandos de verificación dentro de SQLPlus

-- Ver información de la base de datos

```
SELECT name, open_mode FROM v$database;
```

-- Ver versión de Oracle

```
SELECT * FROM v$version WHERE banner LIKE '%Oracle%';
```

-- Salir

EXIT;

PASO 8: CONFIGURACIÓN PARA APLICACIONES EXTERNAS

Datos de conexión:

```
Tipo: Oracle Basic
Host: localhost
Puerto: 1522
Service Name: XE
Usuario: system
Password: 12345
```

8.2 Para SQL Developer:

```
Connection Name: Oracle-XE-Local
Username: system
Password: 12345
Hostname: localhost
Port: 1522
SID: XE
```

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone...	Detalles de Cone...
CURSO	ALINE@//localho...
prueba-oracle	system@//localh...
pruebas2-oracle	system@//localh...
SYSTEM	SYSTEM@//localh...

Nombre:

Tipo de Base de Datos:

Información de usuario Usuario de Proxy

Tipo de autenticación:

Usuario: Rol:

Contraseña: ☐ Guardar Contraseña

Tipo de Conexión:

Detalles Avanzado

Nombre del Host:

Puerto:

☒ SID

☐ Nombre del Servicio

Estado:

URL JDBC:

jdbc:oracle:thin:@localhost:1522:XE

PASO 9: COMANDOS DE GESTIÓN DIARIA

Iniciar/Detener el contenedor

Detener

docker stop oracle-prueba

Iniciar

docker start oracle-prueba

Reiniciar

docker restart oracle-prueba

Ver logs cuando sea necesario

docker logs oracle-prueba --tail 20

Ver consumo de recursos

docker stats oracle-prueba

SOLUCIÓN DE PROBLEMAS

Si el contenedor no inicia

Ver logs detallados

docker logs oracle-prueba

Verificar recursos del sistema

docker system df

Si hay error de puerto

Ver qué proceso usa el puerto 1522

netstat -ano | findstr :1522

Cambiar puerto externo

**docker run -d --name oracle-prueba -p 1523:1521 -e
ORACLE_PASSWORD=12345 gvenzl/oracle-xe:latest**

Reinstalación completa

```
docker stop oracle-prueba
```

```
docker rm oracle-prueba
```

```
docker rmi gvenzl/oracle-xe:latest
```

```
docker system prune -f
```

Volver al PASO 3

Nota: Esto es solo para crear la imagen de Oracle y un contenedor para conectarte. EL contenedor no servirá para conectarte con Spring, deberás apagarlo para que funcione la otra configuración

Conectar Spring Boot con Oracle XE en Docker

1. Crear Docker file

FROM maven:3.8-openjdk-17 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

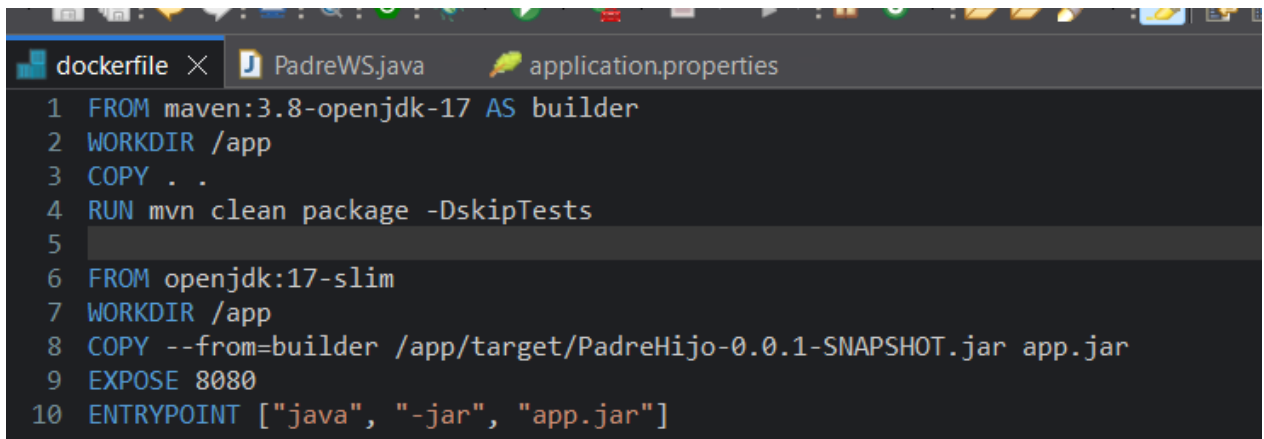
FROM openjdk:17-slim

WORKDIR /app

COPY --from=builder /app/target/PadreHijo-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]



```
1 FROM maven:3.8-openjdk-17 AS builder
2 WORKDIR /app
3 COPY . .
4 RUN mvn clean package -DskipTests
5
6 FROM openjdk:17-slim
7 WORKDIR /app
8 COPY --from=builder /app/target/PadreHijo-0.0.1-SNAPSHOT.jar app.jar
9 EXPOSE 8080
10 ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Construir Jar y crear imagen Docker

1. Navegar a tu proyecto Spring desde powershell:

```
cd "C:\Users\aline\OneDrive\Desktop\CURSO\Semana 5 - Pruebas U,  
Security, Docker\Spring boot\PadreHijo"
```


2. Verificar que el Dockerfile existe:

dir

Directorio: C:\Users\aline\OneDrive\Desktop\CURSO\Semana 5 - Pruebas U, Security, Docker\Spring boot\PadreHijo

Mode	LastWriteTime	Length	Name
d----	29/09/2025 09:43 a. m.		.mvn
d----	29/09/2025 09:43 a. m.		.settings
d----	29/09/2025 09:43 a. m.		src
d----	29/09/2025 09:43 a. m.		target
-a----	30/09/2025 11:47 p. m.	2357	.classpath
-a----	17/09/2025 09:31 a. m.	38	.gitattributes
-a----	17/09/2025 09:31 a. m.	394	.gitignore
-a----	29/09/2025 03:23 p. m.	1021	.project
-a----	01/10/2025 12:28 a. m.	522	docker-compose.yml
-a----	30/09/2025 11:50 p. m.	256	dockerfile
-a----	17/09/2025 09:31 a. m.	1741	HELP.md
-a----	17/09/2025 09:31 a. m.	11790	mvnw
-a----	17/09/2025 09:31 a. m.	8292	mvnw.cmd
-a----	30/09/2025 11:32 p. m.	2587	pom.xml

3. Construir la imagen

docker build -t padre-hijo-spring:1.0 .

4. Modificar el archivo properties de tu proyecto

```
1 spring.application.name=PadreHijo
2
3
4 #puerto
5 server.port=8000
6
7 #configuracion de oracle
8 spring.datasource.url=jdbc:oracle:thin:@oracle-db:1521:XE
9 spring.datasource.username=system
10 spring.datasource.password=12345
11 spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
12 spring.jpa.database=oracle
13 spring.jpa.database-platform=org.hibernate.dialect.OracleDialect
14 spring.jpa.properties.hibernate.formary-sql=true
15
16 logging.level.org.hibernate.sql=debug
17 logging.level.org.hibernate.type.descriptor.sql.BasicBinder=trace
```

Crear docker-compose.yml

Ejecutar en powershell lo siguiente.

```
@"
```

```
services:
```

```
  oracle-db:
```

```
    image: gvenzl/oracle-xe:latest
```

```
    container_name: oracle-db
```

```
    ports:
```

```
      - '1522:1521'
```

```
    environment:
```

```
      - ORACLE_PASSWORD=12345
```

```
    restart: unless-stopped
```

```
  spring-app:
```

```
    image: padre-hijo-spring:1.0
```

```
    container_name: spring-app
```

```
    ports:
```

```
      - '8080:8000' # EXTERNO:INTERNO
```

```
    environment:
```

```
      - SPRING_DATASOURCE_URL=jdbc:oracle:thin:@oracle-db:1521:XE
```

```
      - SPRING_DATASOURCE_USERNAME=system
```

```
      - SPRING_DATASOURCE_PASSWORD=12345
```

```
    restart: unless-stopped
```

```
"@ | Out-File -FilePath "docker-compose.yml" -Encoding UTF8 -Force
```

```

oracle-db:
  image: gvenzl/oracle-xe:latest
  container_name: oracle-db
  ports:
    - '1522:1521'
  environment:
    - ORACLE_PASSWORD=12345
  restart: unless-stopped

```

- Usa la imagen Oracle XE.
- Crea el contenedor con nombre oracle-db.
- Expone el puerto 1521 interno de Oracle en el 1522 de tu máquina local.
- Le pone la contraseña 12345 para el usuario system.
- Se reinicia automáticamente a menos que lo detengas manualmente.

```

spring-app:
  image: padre-hijo-spring:1.0
  container_name: spring-app
  ports:
    - '8080:8000' # EXTERNO:INTERNO
  environment:
    - SPRING_DATASOURCE_URL=jdbc:oracle:thin:@oracle-db:1521:XE
    - SPRING_DATASOURCE_USERNAME=system
    - SPRING_DATASOURCE_PASSWORD=12345
  restart: unless-stopped

```

- Usa tu propia imagen de Spring Boot (padre-hijo-spring:1.0).
- Crea el contenedor con nombre spring-app.
- Expone el puerto 8000 interno en el 8080 de tu máquina local.
- Configura las variables de entorno de Spring DataSource para conectarse a Oracle.
- jdbc:oracle:thin:@oracle-db:1521:XE → aquí oracle-db no es localhost, sino el nombre del servicio/contenedor de Docker (Docker Compose crea una red interna y resuelve ese nombre).
- Usuario/contraseña son system / 12345.
- También se reinicia automáticamente.

Ejecutar docker-compose para crear los contendores

Docker-compose up -d

comando para verificar logs de inicialización:

Monitorear progreso de Oracle

```
docker logs -f oracle-prueba | findstr "READY\|ERROR"
```

Probar que Spring puede conectar a Oracle

```
docker exec spring-app curl http://localhost:8000/actuator/health
```

Crea la nueva conexión con SQL

Nueva / Seleccionar Conexión a Base de Datos

Nombre de Cone...	Detalles de Cone...
CURSO	ALINE@//localho...
prueba-oracle	system@//localh...
pruebas2-oracle	system@//localh...
SYSTEM	SYSTEM@//localh...

Name:

Tipo de Base de Datos:

Información de usuario: Usuario de Proxy

Tipo de autenticación:

Usuario: Rol:

Contraseña:

Guardar Contraseña: ☐

Tipo de Conexión:

Detalles:

Nombre del Host:

Puerto:

☒ SID:

☐ Nombre del Servicio:

Estado:

Ayuda Guardar Borrar Probar Conectar Cancelar

Crea las tablas correspondientes

```
CREATE TABLE padre (
    id_padre NUMBER PRIMARY KEY,
    nombre VARCHAR2(100) NOT NULL,
    apellido VARCHAR2(100),
    edad NUMBER,
    trabajo VARCHAR2(100)
);

CREATE TABLE hijo (
    id_hijo NUMBER PRIMARY KEY,
    nombre VARCHAR2(100) NOT NULL,
    apellido VARCHAR2(100),
    edad NUMBER,
    hobbie VARCHAR2(100),
    padre_id number,
    FOREIGN KEY (padre_id) REFERENCES padre(id_padre)
);
```

```
CREATE TABLE padre (  
    id_padre NUMBER PRIMARY KEY,  
    nombre VARCHAR2(100) NOT NULL,  
    apellido VARCHAR2(100),  
    edad NUMBER,  
    trabajo VARCHAR2(100)  
);
```

```
CREATE TABLE hijo (  
    id_hijo NUMBER PRIMARY KEY,  
    nombre VARCHAR2(100) NOT NULL,  
    apellido VARCHAR2(100),  
    edad NUMBER,  
    hobby VARCHAR2(100),  
    padre_id number,  
    FOREIGN KEY (padre_id) REFERENCES padre(id_padre)  
);
```

-- Comando para verificar tablas en Oracle

```
SELECT table_name FROM user_tables;
```

Script de prueba de endpoints:

script de prueba curl

```
curl -X GET http://localhost:8080/api/Padre/listar
```

En postman

```
GET http://localhost:8080/api/Padre/listar
```

Comandos de gestión diaria para ambos contenedores:

Ver todos los contenedores

docker-compose ps

#Reiniciar solo Spring

docker-compose restart spring-app

Ver logs combinados

docker-compose logs -f